

Claw GUI Interface

Generated by Doxygen 1.16.0

1 README	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Class Documentation	9
5.1 AppDelegate Class Reference	9
5.1.1 Detailed Description	10
5.1.2 Method Documentation	10
5.1.2.1 BumpOriginDownButtonPressed:	10
5.1.2.2 connectButtonPressed:	10
5.1.2.3 enableButtonPressed:	10
5.1.2.4 enableCalibrationButtonPressed:	10
5.1.2.5 PositionSliderMoved:	11
5.1.2.6 readStatusButtonPressed:	11
5.1.2.7 setOriginButtonPressed:	11
5.1.2.8 speedSliderMoved:	11
5.1.2.9 stopClawButtonPressed:	12
5.2 AppDelegate() Category Reference	12
5.3 clawControl Class Reference	13
5.3.1 Method Documentation	14
5.3.1.1 bumpClawZeroDownWithError:	14
5.3.1.2 connectToSerialPortWithError:	14
5.3.1.3 disableClawStepperWithError:	14
5.3.1.4 disconnectFromSerialPortWithError:	15
5.3.1.5 enableClawStepperWithError:	15
5.3.1.6 readStatusToDictionaryWithError:	15
5.3.1.7 setClawErrorWithCode:andDescription:andReason:andSuggestion:	16
5.3.1.8 setClawPosition:withError:	16
5.3.1.9 setClawSpeed:withError:	17
5.3.1.10 setClawZeroWithError:	17
5.3.1.11 setDevicePathWith:withError:	17
5.3.1.12 stopClawMotionWithError:	18
5.4 UART Struct Reference	18
5.4.1 Detailed Description	19
6 File Documentation	21
6.1 AppDelegate.h	21

6.2 ClawGUI/clawControl.h File Reference	21
6.2.1 Detailed Description	22
6.3 clawControl.h	22
6.4 ClawGUI/clawControl.m File Reference	23
6.4.1 Detailed Description	24
6.5 ClawGUI/main.m File Reference	24
6.5.1 Detailed Description	25
6.5.2 Function Documentation	25
6.5.2.1 main()	25
Index	27

Chapter 1

README

ClawGUI is a GUI interface to a Raspberry Pico running the claw firmware. ClawGUI is a MacOS application written in Objective-C. This should compile on a stock Xcode install.

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AppDelegate()	12
<UIApplicationDelegate>	
AppDelegate	9
NSObject	
AppDelegate	9
clawControl	13
UART	18

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AppDelegate		9
AppDelegate()		12
clawControl		13
UART		
Structure to hold UART data and state, include termios options		18

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

ClawGUI/ AppDelegate.h	21
ClawGUI/ clawControl.h	
Definition of claw class	21
ClawGUI/ clawControl.m	
Implimentation of claw class	23
ClawGUI/ main.m	
Definition of claw class	24

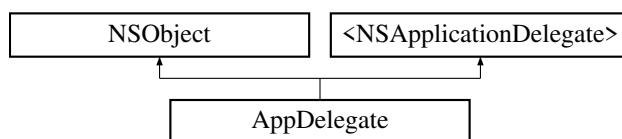
Chapter 5

Class Documentation

5.1 AppDelegate Class Reference

```
#include <AppDelegate.h>
```

Inheritance diagram for AppDelegate:



Instance Methods

- (IBAction) - [connectButtonPressed:](#)
Handle connect button pressd action.
- (IBAction) - [enableButtonPressed:](#)
Handle enable button pressd action.
- (IBAction) - [PositionSliderMoved:](#)
Handle claw position slider moved action.
- (IBAction) - [stopClawButtonPressed:](#)
Handle read status button press.
- (IBAction) - [readStatusButtonPressed:](#)
Handle read status button press.
- (IBAction) - [speedSliderMoved:](#)
Handle claw speed slider moved action.
- (IBAction) - [enableCalibrationButtonPressed:](#)
Handle enable calibration button press.
- (IBAction) - [setOriginButtonPressed:](#)
Handle Set Origin button press.
- (IBAction) - [BumpOriginDownButtonPressed:](#)
Handle Set Origin button press.

5.1.1 Detailed Description

Define [AppDelegate](#) for GUI. This delegate handles all GUI actions from main window

5.1.2 Method Documentation

5.1.2.1 BumpOriginDownButtonPressed:

```
- (IBAction) BumpOriginDownButtonPressed:
    (id) sender
```

Handle Set Origin button press.

Handle Bump Down button press. Issues command to bump origin down a quarter turn and reset origin to new origin location (-1/4 turn). Used to set calibrated zero point of claw device

Parameters

<i>sender</i>	Sender object of action
---------------	-------------------------

5.1.2.2 connectButtonPressed:

```
- (IBAction) connectButtonPressed:
    (id) sender
```

Handle connect button pressed action.

Handle connect button pressed action. Connects to claw device over [UART](#). Also enables Enable button. Changes button label to "Disconnect" on connect. When in Disconnect state, it will disconnect from claw device.

Parameters

<i>sender</i>	Sender object of action
---------------	-------------------------

5.1.2.3 enableButtonPressed:

```
- (IBAction) enableButtonPressed:
    (id) sender
```

Handle enable button pressed action.

Handle enable button pressed action. Enables stepper motor of claw. Changes label to "Disable" on success. Also enables controls such as position slider, speed slider, stop button and enable cal button

Parameters

<i>sender</i>	Sender object of action
---------------	-------------------------

5.1.2.4 enableCalibrationButtonPressed:

Parameters

<i>sender</i>	Sender object of action
---------------	-------------------------

5.1.2.5 PositionSliderMoved:

```
- (IBAction) PositionSliderMoved:
    (id) sender
```

Handle claw position slider moved action.

Handle claw position slider movement. Issues command to claw device to move claw to mirror slider position

Parameters

<i>sender</i>	Sender object of action
---------------	-------------------------

5.1.2.6 readStatusButtonPressed:

```
- (IBAction) readStatusButtonPressed:
    (id) sender
```

Handle read status button press.

Handle read status button press. Issues command to claw device to return status. Updates sliders based on read status. Usefull if an external E-Stop occures to update slider positions.

Parameters

<i>sender</i>	Sender object of action
---------------	-------------------------

5.1.2.7 setOriginButtonPressed:

```
- (IBAction) setOriginButtonPressed:
    (id) sender
```

Handle Set Origin button press.

Handle Set Origin button press. Issues command to reset origin of claw to current position

Parameters

<i>sender</i>	Sender object of action
---------------	-------------------------

5.1.2.8 speedSliderMoved:

```
- (IBAction) speedSliderMoved:
    (id) sender
```

Handle claw speed slider moved action.

Parameters

<i>sender</i>	Sender object of action
---------------	-------------------------

5.1.2.9 stopClawButtonPressed:

```
- (IBAction) stopClawButtonPressed:
    (id) sender
```

Handle read status button press.

Handle stops button press. Issues command to claw device to stop motion. Updates position slider based on read status.

Parameters

<i>sender</i>	Sender object of action
---------------	-------------------------

The documentation for this class was generated from the following files:

- ClawGUI/AppDelegate.h
- ClawGUI/AppDelegate.m

5.2 AppDelegate() Category Reference**Protected Attributes**

- bool **serialPortConnected**
Member variable to keep track of serial port connection state.
- bool **calibrationModeEnabled**
Member variable to keep track of calibration mode.

Properties

- **clawControl** * **claw**
Claw object, used to communicate with claw and issue commands to claw device.
- IBOutlet NSWindow * **window**
Main GUI Window.
- IBOutlet NSComboBox * **serialPortComboBox**
Combo box with drop down list of available serial port devices ("/dev/cu.usbmodem").*
- IBOutlet NSButton * **connectButton**
Button to connect or disconnect from claw device.
- IBOutlet NSButton * **enableClawButton**
Button to enable stepper motor on claw device.
- IBOutlet NSSlider * **clawPositionSlider**
Slider to set position of claw. Also position is moved on a STOP or READ STATUS.
- IBOutlet NSButton * **stopClawButton**

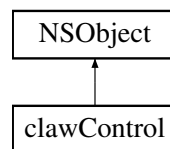
- Button to stop claw motion immediately, somewhat liek an emergency stop, but in software.
- IBOutlet NSButton * **readStatusButton**
Button to read status of claw device. Usefull to update position slider on external E-Stop.
- IBOutlet NSSlider * **clawSpeedSlider**
Slider to set claw speed, specifically stepper pulse rate, between 2.5kHz and 25kHz.
- IBOutlet NSButton * **enableCalibrationButton**
Enables calibration buttons, specifically "Set Origin" and "Bump Down" Buttons.
- IBOutlet NSButton * **setOriginButton**
Button to set claw origin to current location.
- IBOutlet NSButton * **bumpOriginDownButton**
Button to bump down calibrated zero by 1/4 turn to move zero point down a little bit.

The documentation for this category was generated from the following file:

- ClawGUI/AppDelegate.m

5.3 clawControl Class Reference

Inheritance diagram for clawControl:



Instance Methods

- (NSError *) - [setClawErrorWithCode:andDescription:andReason:andSuggestion:](#)
Creates and NSError with appropriate keys for Claw objects.
- (BOOL) - [setDevicePathWith:error:](#)
Sets device descriptor path to serial port.
- (BOOL) - [connectToSerialPortWith:error:](#)
Connects the claw device to the serial port specified with setDevicePathWith:
- (BOOL) - [disconnectFromSerialPortWith:error:](#)
Disconnects the claw device from the serial port if already connected.
- (BOOL) - [enableClawStepperWith:error:](#)
Sets claw device to enabled.
- (BOOL) - [disableClawStepperWith:error:](#)
sets claw device to disabled
- (BOOL) - [setClawPosition:with:error:](#)
sets claw position to parameter clawPosition (0 to 100)
- (BOOL) - [stopClawMotionWith:error:](#)
stops claw motion immediately, regardless of current position
- (NSDictionary *) - [readStatusToDictionaryWith:error:](#)
reads status from claw device and returns dictionary of status
- (BOOL) - [setClawSpeed:with:error:](#)
Set claw speed expressed as stepper period.
- (BOOL) - [setClawZeroWith:error:](#)
sets claw zero position to current position. Used to calibrate zero on claw
- (BOOL) - [bumpClawZeroDownWith:error:](#)
bumps zero and moves stepper 1/4 turn below current zero to bump down zero set point

Protected Attributes

- dispatch_semaphore_t **serialPortSemaphore**

Semaphore for serial port access – NOT IMPLEMENTED, Need to implement for threaded operation.

Properties

- BOOL **connectionStatus**

The status of the serial connection to the Claw controller.

- BOOL **clawStepperEnabled**

The state of the stepper in the claw, Enabled = TRUE, Disabled = False.

- NSString * **devicePath**

The path to the device descriptor in the /dev tree for the serial connection to the Claw controller.

5.3.1 Method Documentation

5.3.1.1 bumpClawZeroDownWithError:

```
- (BOOL) bumpClawZeroDownWithError:
    (NSError**) error
```

bumps zero and moves stepper 1/4 turn below current zero to bump down zero set point

Parameters

<i>error</i>	pointer to NSError* on failure to send bump down command, nil otherwise
--------------	---

Returns

EXIT_FAILURE on fail, EXIT_SUCCESS on success

5.3.1.2 connectToSerialPortWithError:

```
- (BOOL) connectToSerialPortWithError:
    (NSError**) error
```

Connects the claw device to the serial port specified with setDevicePathWith:

Connects to device set with setDevicePathWith: method. This also turns off echo on the firmware interface and clears the serial RX buffer. This makes the claw device firmware and connection ready to execute commands

Parameters

<i>error</i>	pointer to NSError* on failure to connect to the Device, nil otherwise
--------------	--

Returns

EXIT_FAILURE on fail, EXIT_SUCCESS on success

Parameters

<i>error</i>	pointer to NSError* on failure to disable the claw, nil otherwise
--------------	---

Returns

EXIT_FAILURE on fail, EXIT_SUCCESS on success

5.3.1.4 disconnectFromSerialPortWithError:

```
- (BOOL) disconnectFromSerialPortWithError:
    (NSError**) error
```

Disconnects the claw device from the serial port if already connected.

Disconnects from the claw serial port and frees the file handle

Parameters

<i>error</i>	pointer to NSError* on failure to disconnect to the serial port of the claw device
--------------	--

Returns

returns NSError on failure to disconnect and free file handle

5.3.1.5 enableClawStepperWithError:

```
- (BOOL) enableClawStepperWithError:
    (NSError**) error
```

Sets claw device to enabled.

Sets claw stepper motor to enabled and ready to move claw device

Parameters

<i>error</i>	pointer to NSError* on failure to enable the claw, nil otherwise
--------------	--

Returns

EXIT_FAILURE on fail, EXIT_SUCCESS on success

5.3.1.6 readStatusToDictionaryWithError:

```
- (NSDictionary *) readStatusToDictionaryWithError:
    (NSError**) error
```

Generated by Doxygen

reads status from claw device and returns dictionary of status

Status of claw device key Current Position : current claw position in number of steps key Target Position : target

Parameters

<i>error</i>	pointer to NSError* on failure to get status
--------------	--

Returns

NSDictionary pointer to dictionary of current status

5.3.1.7 setClawErrorWithCode:andDescription:andReason:andSuggestion:

```
- (NSError *) setClawErrorWithCode:
    (NSInteger) errorCode
    andDescription:(NSString*) errorDescription
    andReason:(NSString*) errorReason
    andSuggestion:(NSString*) errorSuggestion
```

Creates and NSError with appropriate keys for Claw objects.

Parameters

<i>errorCode</i>	NSInteger error code for error
<i>errorDescription</i>	NSString pointer to description of error
<i>errorReason</i>	NSString pointer to the reason the error occurred
<i>errorSuggestion</i>	NSString pointer to a suggestion for the user to rectify the error

Returns

: returns NSError created or nil if failed to create error

5.3.1.8 setClawPosition:withError:

```
- (BOOL) setClawPosition:
    (NSInteger) clawPosition
    withError:(NSError**) error
```

sets claw position to parameter clawPosition (0 to 100)

Sets claw position based on clawPosition parameter to a value between 0 and 100

Parameters

<i>clawPosition</i>	NSInteger position
<i>error</i>	pointer to NSError* on failure set the claw position, nil otherwise

Returns

EXIT_FAILURE on fail, EXIT_SUCCESS on success

5.3.1.9 setClawSpeed:withError:

```
- (BOOL) setClawSpeed:
    (NSInteger) clawPeriod
    withError:(NSError**) error
```

Set claw speed expressed as stepper period.

Note

Claw speed is expressed as a period in microseconds

Sets claw speed in microseconds per step. Reasonable speeds are between 40µs and 400µs (25kHz to 2.5kHz)

Parameters

<i>clawPeriod</i>	microsecond period to set claw stepper motor pulse rate
<i>error</i>	pointer to NSError* on failure to get status

Returns

NSDictionary pointer to dictionary of current status

5.3.1.10 setClawZeroWithError:

```
- (BOOL) setClawZeroWithError:
    (NSError**) error
```

sets claw zero position to current position. Used to calibrate zero on claw

Parameters

<i>error</i>	pointer to NSError* on failure to send set zero command, nil otherwise
--------------	--

Returns

EXIT_FAILURE on fail, EXIT_SUCCESS on success

5.3.1.11 setDevicePathWith:withError:

```
- (BOOL) setDevicePathWith:
    (NSString*) pathToSet
    withError:(NSError**) error
```

Sets device descriptor path to serial port.

Parameters

<i>pathToSet</i>	NSString pointer to device path of serial port
<i>error</i>	pointer to NSError* on failure to set device path, nil otherwise

Returns

NSError if pathToSet parameter is nil

5.3.1.12 stopClawMotionWithError:

```
- (BOOL) stopClawMotionWithError:
    (NSError**) error
```

stops claw motion immediately, regardless of current position

Parameters

<i>error</i>	pointer to NSError* on failure to send stop command, nil otherwise
--------------	--

Returns

EXIT_FAILURE on fail, EXIT_SUCCESS on success

The documentation for this class was generated from the following files:

- ClawGUI/[clawControl.h](#)
- ClawGUI/[clawControl.m](#)

5.4 UART Struct Reference

Structure to hold [UART](#) data and state, include termios options.

Public Attributes

- int **uartFileHandle**
File handle for [UART](#) device descriptor.
- char * **uartPath**
Absolute path to [UART](#) file descriptor, should be in /dev/.
- int **uartBaud**
[UART](#) Baud Rate.
- int **uartParity**
[UART](#) Parity (Even/Odd/None).
- int **uartStopBits**
[UART](#) Stop Bits.
- int **uartByteSize**
[UART](#) Byte Size, should be 8.
- struct termios **uartOptions**
termios [UART](#) Options, This is where the actual options get stored.

5.4.1 Detailed Description

Structure to hold [UART](#) data and state, include termios options.

The documentation for this struct was generated from the following file:

- [ClawGUI/clawControl.m](#)

Chapter 6

File Documentation

6.1 AppDelegate.h

```
00001
00012
00013 #import <Cocoa/Cocoa.h>
00014
00018 @interface AppDelegate : NSObject <NSApplicationDelegate>
00019
00026 - (IBAction)connectButtonPressed:(id) sender;
00027
00034 - (IBAction)enableButtonPressed:(id) sender;
00035
00041 - (IBAction)positionSliderMoved:(id) sender;
00042
00049 - (IBAction)stopClawButtonPressed:(id) sender;
00050
00057 - (IBAction)readStatusButtonPressed:(id) sender;
00058
00065 - (IBAction)speedSliderMoved:(id) sender;
00066
00073 - (IBAction)enableCalibrationButtonPressed:(id) sender;
00074
00080 - (IBAction)setOriginButtonPressed:(id) sender;
00081
00088 - (IBAction)bumpOriginDownButtonPressed:(id) sender;
00089
00090 @end
00091
```

6.2 ClawGUI/clawControl.h File Reference

definition of claw class

```
import <Foundation/Foundation.h>
```

Classes

- class [clawControl](#)

Macros

- **#define CLAW_MAX_PERIOD 400**
Maximum claw step period Freq = 1 / period = 2.5kHz.
- **#define CLAW_MIN_PERIOD 40**
Minimum claw step period Freq = 1 / period = 25kHz.
- **#define CLAW_ERROR_DOMAIN @"com.jon-wade.ClawGUI.SerialError"**
NSError domain for errors created in clawControl.c.
- **#define CLAW_ERROR_INVALID_PORT_PATH 100**
Error code for invalid serial port path.
- **#define CLAW_ERROR_NO_DATA_TO_READ 101**
Error code for no data in receive queue of serial port.
- **#define CLAW_ERROR_STEPPER_NOT_CONNECTED 200**
Error code for claw controller not connected to serial port.
- **#define CLAW_ERROR_STEPPER_NOT_ENABLED 201**
Error code for claw controller stepper motor not connected.
- **#define CLAW_ERROR_INCORRECT_RESPONSE 202**
Error code for incorrect response to command, specifically status command.

6.2.1 Detailed Description

definition of claw class

Author

Jon Wade

Date

24 Dec 2025

Copyright

(c) 2025 Jon Wade. Standard MIT License applies. See LICENSE file.

This file contains the definition of the class for controlling claw over the serial port including connecting and disconnecting the serial port.

6.3 clawControl.h

[Go to the documentation of this file.](#)

```
00001
00012 #import <Foundation/Foundation.h>
00013
00014
00015 #define CLAW_MAX_PERIOD 400
00016 #define CLAW_MIN_PERIOD 40
00017
00018 #define CLAW_ERROR_DOMAIN @"com.jon-wade.ClawGUI.SerialError"
00019 #define CLAW_ERROR_INVALID_PORT_PATH 100
00020 #define CLAW_ERROR_NO_DATA_TO_READ 101
00021 #define CLAW_ERROR_STEPPER_NOT_CONNECTED 200
00022 #define CLAW_ERROR_STEPPER_NOT_ENABLED 201
```

```

00023 #define CLAW_ERROR_INCORRECT_RESPONSE      202
00024
00025 NS_ASSUME_NONNULL_BEGIN
00026
00027 @interface clawControl : NSObject
00028 {
00029     dispatch_semaphore_t serialPortSemaphore;
00030 }
00031
00035 @property BOOL connectionStatus;
00036
00040 @property BOOL clawStepperEnabled;
00041
00045 @property (strong) NSString* devicePath;
00046
00056 - (NSError*) setClawErrorWithCode: (NSInteger) errorCode
00057                andDescription: (NSString*) errorDescription
00058                andReason: (NSString*) errorReason
00059                andSuggestion: (NSString*) errorSuggestion;
00060
00068 - (BOOL) setDevicePathWith: (NSString*) pathToSet withError: (NSError**) error;
00069
00080 - (BOOL) connectToSerialPortWithError: (NSError**) error;
00081
00090 - (BOOL) disconnectFromSerialPortWithError: (NSError**) error;
00091
00100 - (BOOL) enableClawStepperWithError: (NSError**) error;
00101
00110 - (BOOL) disableClawStepperWithError: (NSError**) error;
00111
00121 - (BOOL) setClawPosition: (NSInteger) clawPosition withError: (NSError**) error;
00122
00129 - (BOOL) stopClawMotionWithError: (NSError**) error;
00130
00146 - (NSDictionary*) readStatusToDictionaryWithError: (NSError**) error;
00147
00159 - (BOOL) setClawSpeed: (NSInteger) clawPeriod withError: (NSError**) error;
00160
00167 - (BOOL) setClawZeroWithError: (NSError**) error;
00168
00175 - (BOOL) bumpClawZeroDownWithError: (NSError**) error;
00176
00177 @end
00178
00179 NS_ASSUME_NONNULL_END

```

6.4 ClawGUI/clawControl.m File Reference

implimentation of claw class

```

import "clawControl.h"
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <termios.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/fcntl.h>

```

Classes

- struct [UART](#)

Structure to hold [UART](#) data and state, include termios options.

Typedefs

- typedef struct [UART](#) **UART_t**
typedef of [UART](#) structure

Variables

- [UART_t](#) **myUART**
[UART_t](#) structure to hold [UART](#) Data.

6.4.1 Detailed Description

implimentation of claw class

Author

Jon Wade

Date

24 Dec 2025

Copyright

(c) 2025 Jon Wade. Standard MIT License applies. See LICENSE file.

This file contains the implimentation of the class for controlling claw over the serial port including connecting and disconnecting the serial port.

6.5 ClawGUI/main.m File Reference

definition of claw class

```
import <Cocoa/Cocoa.h>
```

Functions

- int [main](#) (int argc, const char *argv[])
Standard boilerplate main definition for most Cocoa applications.

6.5.1 Detailed Description

definition of claw class

Author

Jon Wade

Date

23 Dec 2025

Copyright

(c) 2025 Jon Wade. Standard MIT License applies. See LICENSE file.

This file contains the main function of ClawGUI. This is a standard boilerplate main for most Cocoa framework applications

6.5.2 Function Documentation

6.5.2.1 main()

```
int main (  
    int argc,  
    const char * argv[])
```

Standard boilerplate main definition for most Cocoa applications.

Parameters

<i>argc</i>	Argument count for main... typically not used in cocoa applications but passed to NSApplicationMain
<i>argv</i>	Array of strings of arguments... typically not used in cocoa applications but passed to NSApplicationMain

Returns

returns return value of NSApplicationMain

Index

AppDelegate, 9

- BumpOriginDownButtonPressed:, 10
- connectButtonPressed:, 10
- enableButtonPressed:, 10
- enableCalibrationButtonPressed:, 10
- PositionSliderMoved:, 11
- readStatusButtonPressed:, 11
- setOriginButtonPressed:, 11
- speedSliderMoved:, 11
- stopClawButtonPressed:, 12

AppDelegate(), 12

bumpClawZeroDownWithError:

- clawControl, 14

BumpOriginDownButtonPressed:

- AppDelegate, 10

clawControl, 13

- bumpClawZeroDownWithError:, 14
- connectToSerialPortWithError:, 14
- disableClawStepperWithError:, 14
- disconnectFromSerialPortWithError:, 15
- enableClawStepperWithError:, 15
- readStatusToDictionaryWithError:, 15
- setClawErrorWithCode:andDescription:andReason:andSuggestion:, 16
- setClawPosition:withError:, 16
- setClawSpeed:withError:, 16
- setClawZeroWithError:, 17
- setDevicePathWith:withError:, 17
- stopClawMotionWithError:, 18

ClawGUI/AppDelegate.h, 21

ClawGUI/clawControl.h, 21, 22

ClawGUI/clawControl.m, 23

ClawGUI/main.m, 24

connectButtonPressed:

- AppDelegate, 10

connectToSerialPortWithError:

- clawControl, 14

disableClawStepperWithError:

- clawControl, 14

disconnectFromSerialPortWithError:

- clawControl, 15

enableButtonPressed:

- AppDelegate, 10

enableCalibrationButtonPressed:

- AppDelegate, 10

enableClawStepperWithError:

- clawControl, 15

main

- main.m, 25

main.m

- main, 25

PositionSliderMoved:

- AppDelegate, 11

README, 1

readStatusButtonPressed:

- AppDelegate, 11

readStatusToDictionaryWithError:

- clawControl, 15

setClawErrorWithCode:andDescription:andReason:andSuggestion:

- clawControl, 16

setClawPosition:withError:

- clawControl, 16

setClawSpeed:withError:

- clawControl, 16

setClawZeroWithError:

- clawControl, 17

setDevicePathWith:withError:

- clawControl, 17

setOriginButtonPressed:

- AppDelegate, 11

speedSliderMoved:

- AppDelegate, 11

stopClawButtonPressed:

- AppDelegate, 12

stopClawMotionWithError:

- clawControl, 18

UART, 18